Path planning of mobile robot based on improved particle swarm optimization algorithm

Naigong Yu*
Faculty of Information Technology
Beijing University of Technology
Beijing, China
yunaigong@bjut.edu.cn

Tong Qiu
Faculty of Information Technology
Beijing University of Technology
Beijing, China
q1297014133@163.com

Jinhan Yan
Faculty of Information Technology
Beijing University of Technology
Beijing, China
yanjinhan@emails.bjut.edu.cn

***Abstract*—An enhanced algorithm for optimizing the motion path of mobile robots in static environments is presented in this paper. The proposed algorithm, named as Improved Particle Swarm Optimization (IPSO), utilizes advanced techniques to enhance the traditional particle swarm optimization (PSO) approach, which addresses the issues of premature global optimization and unsatisfactory path accuracy when using traditional PSO. Firstly, consider the path length and collision risk as optimization objectives to improve the efficiency and safety of the robot's movement. Then, without compromising the fast convergence characteristics of the PSO algorithm, the inertia weight parameter of the algorithm are dynamically adjusted based on the current position of the particles, balancing the exploration of global or local regions in the environmental space. Finally, the proposed algorithm is applied to different experimental scenarios containing multiple static obstacles for path planning simulation experiments. The MATLAB simulation results show that compared with traditional PSO algorithm, the IPSO algorithm proposed in this paper has faster convergence velocity in complex environments, shorter planned path length, and stronger search ability. This study provides a practical and feasible solution for mobile robots to optimize their motion paths in static environments. It is expected to achieve wide application in robot path planning, improve the navigation and path planning ability of mobile robots, and promote the development of intelligent robot technology.**

Keywords—mobile robot, Path planning, particle swarm optimization, static environment

I. INTRODUCTION

With the advancement of robotics technology, mobile robots have become increasingly prevalent in a variety of areas in our daily lives [1]. In these practical applications, navigation technology is the core of reflecting the intelligent ability of mobile robots, and path planning holds considerable significance in the field of navigation research as it serves as a crucial aspect and subject of study. Path Planning is a process of finding the optimal or feasible path from a starting point to the destination point in a given environment or space [2]. The application of advanced path planning technology to mobile robots can enable them to explore and navigate harsh and inaccessible environments where human presence may not be feasible, as well as in the field of intelligent warehousing, which can reduce human and material resources, and improve the overall transportation efficiency. It is a very meaningful research topic. In the past decades, with the continuous progress of artificial intelligence research, numerous smart algorithms have been employed in mobile robot path planning as well, including fuzzy logic and reinforcement learning algorithm, cell decomposition method, neural network, etc. Path planning algorithms can be divided into two categories. The first category consists of classical algorithms such as artificial potential field method [3], visual graph method [4], RRT algorithm [5], etc. The other is intelligent algorithm, including neural network algorithm [6], particle swarm algorithm [7], etc. Because the calculation takes a long time and cannot deal with the accidental situation in the environment, the classical method is not often used for real-time mobile path planning. Compared with classical algorithms, intelligent algorithms can usually find better paths faster when facing complex maps.

Supported by National Nature Science Foundation of China No.62076-014, Beijing Municipal Science & Technology Commission, Administrative Commission of Zhongguancun Science Park No. Z221100000222016.

II. RELATED WORK

Particle swarm optimization (PSO) is an intelligent method invented by Dr. Kennedy and Professor Eberhart by observing the predatory behavior of group animals. The local optimization generates the global optimal solution through the interaction between certain search rules and individuals. Due to the advantages of having fewer parameters to configure and having a relatively simple mathematical model that does not involve complex mathematical calculations, the PSO is originally employed in addressing non-linear, continuous optimization issues, with one of its main applications is path planning for robots. Nonetheless, conventional PSO has problems such as premature convergence and long optimization time in complex maps. To effectively address these issues, a variety of approaches and concepts have put forward different methods and ideas. Zhang et al. [8] adopted an elite strategy to update particles, which can quickly obtain paths that meet the requirements. But elite strategies limit population diversity. Mandava et al. [9] integrated the potential field method for stationary obstacles with particle swarm optimization to enhance path smoothness. Despite these efforts, the resulting path's length and smoothness are not optimal. Oleiwi [10] developed a hybrid method that combines ant colony optimization algorithm with GA. The ant colony optimization algorithm is used to obtain the barrier-free suboptimal path, which is then used as the initial path of GA.

In order to address the issues of long initial map exploration time, premature convergence, and getting stuck in local optima for the basic PSO in path planning, this paper presents an improved particle swarm optimization (IPSO) approach based on adaptive inertia weight. The purpose of this algorithm is to assist mobile robots in navigating through environments filled with static obstacles of varying sizes by generating collision-free global paths that extend from the starting point to the intended destination. This method uses the distance between particles and obstacles as the feedback signal of the algorithm to dynamically adjust the inertia weight to optimize the PSO algorithm. It solves low convergence accuracy and premature convergence problems caused by the imbalance of global and local search in the complex environment of the traditional algorithm, so that the

2023 International Conference on Service Robotics (ICoSR) | 979-8-3503-0468-8/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICOSR59980.2023.00016

979-8-3503-0468-8/23/$31.00 ©2023 IEEE
DOI 10.1109/ICoSR59980.2023.00016

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 06:41:09 UTC from IEEE Xplore. Restrictions apply.

algorithm can quickly and safely plan a collision-free optimal path.

III. METHOD

A. Standard Particle Swarm Optimization

In the PSO system, the candidate solution set for optimization problems is defined as a particle swarm that may define trajectories through search space. Each alternative solution is referred to as a particle. Particles in the search area continuously strive towards achieving optimal solutions by leveraging not only their own past experiences but also learning from the best experiences of nearby particles, which leads to improved positions. Each particle in the PSO has a position and velocity. Position represents the location of the particle in the space, while velocity is the rate of change from the current position to the next position. Assuming that the entire particle swarm contains N particles, the dimension of each particle is D, and the location of the ith particle is described as $X_i = (x_1, x_2, \dots, x_N)$, the velocity of the ith particle is described as $V_i = (v_1, v_2, \dots, v_N)$. The PSO algorithm can be described as formula (1) and formula (2). Fig. 1 is the principle diagram of the PSO iteration.

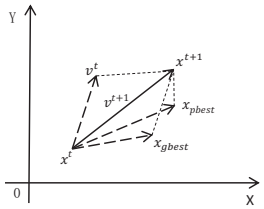


Fig. 1. The principle diagram of particle swarm optimization iteration

Particle velocity update:

$$v_i^d(t+1) = \omega v_i^d(t) + c_1 r_1 \left(x_{pbest_i}^d - x_i^d(t)\right) + c_2 r_2 \left(x_{gbest_i}^d - x_i^d(t)\right) \quad (1)$$

Particle position update:

$$x_i^d(t+1) = x_i^d(t) + v_i^d(t+1) \quad (2)$$

Where, *i=1,2... N*; $v_i^d(t)$ represents the d-dimensional component of the velocity vector of particle *i* in the t-th iteration; $x_i^d(t)$ represents the d-dimensional component of the position vector of particle *i*; $x_{pbest_i}^d$ denotes the d-dimensional component of optimal position pbest of particle *i*; $x_{gbest_i}^d$ denotes the d-dimensional component of optimal position gbest of the whole particle swarm; c_1、c_2 represents the acceleration factor, usually selected as 2; r_1 and r_2 are randomly generated values between 0 and 1; ω is the inertia weight. The velocity and position of particles are randomly generated and updated according to formula (1) and formula (2) until the optimal solution is obtained. The velocity of each one-dimensional particle is limited to the maximum velocity V_{max}. If the updated velocity of each one-dimensional particle exceeds the set V_{max}, the velocity will be limited to V_{max}.

B. Fitness Function

Path planning aims to identify a path curve that satisfies multiple constraints. Therefore, fitness function is a very important factor in particle swarm optimization algorithm, which determines whether the algorithm can converge. Based on the actual demand, the evaluation index mainly includes two factors: safety degree and path length. Therefore, this paper takes the shortest path and obstacle collision risk as the criteria for quantitative path finding. Setting that the coordinate of the starting point S as P_1, and the coordinate of the target point G as P_n. The generated path is composed of $Path = \{P_1, P_2, \dots, P_n\}$ means that the path point P_i is a non-obstacle point, and the line between P_i and adjacent points P_{i-1} and P_{i+1} cannot pass through obstacles.

In the path length part, the shortest path means to minimize the path length from the starting position to the target position. The Euclidean distance is used to calculate the length of two position coordinates, as shown in equation (3).

$$l(i) = \sqrt{(a(i+1) - a(i))^2 + (b(i+1) - b(i))^2} \quad (3)$$

Where $(a(i),\ b(i))$ and $(a(i+1),\ b(i+1))$ represent the position coordinates of the path point P_i and P_{i+1} respectively.

Then the distance between the starting point S and the target point G that the mobile robot needs to travel along a given path can be represented by formula (4).

$$f_1 = \sum_{i=1}^{n-1} l(i) \quad (4)$$

The most important goal of the robot is that the moving path cannot contact with the obstacles in the working environment and maintain a certain safe distance. Therefore, it is necessary to determine whether the obtained route intersects with obstacles or whether the distance is too close. Circles are utilized in the environment model to represent any obstacles present. To achieve a path that avoids collisions, a critical factor in ensuring safety along the path is maintaining a distance *d* greater than the safety threshold between both the path node and its corresponding line, and any obstacles. For nodes that pass through obstacles, they are judged as dangerous and punished. The punishment coefficient can be calculated by formula (5) and formula (6).

$$d_i = min\left[|x_i - X_{obs}|, |y_i - Y_{obs}|\right] \quad (5)$$

$$\alpha_i = \begin{cases} 1 & d_i \le R \\ \frac{1}{d_i} & R < d_i < d_{safe} \\ 0 & d_i \ge d_{safe} \end{cases} \quad (6)$$

Where, the variable d_i indicates the measure of the separation between the particle i and the center of the obstruction; (X_{obs}, Y_{obs}) is the center coordinate of the obstacle in the map. The closer the center of the distance to the obstacle, the more unsafe the path is; α_i is the risk factor, representing the size of punishment; R denotes the radius of obstacles; d_{safe} is the security threshold. Formula (6) means that the penalty will be accepted within the safety threshold of the obstacle. The greater the penalty

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 06:41:09 UTC from IEEE Xplore. Restrictions apply.

is near the center, and the penalty will not be punished if it is not within the effective range of the obstacle.

In order to determine whether the path is reliable, formula (7) is used to calculate the collision risk of the path.

$$f_2 = \theta \sum_{i=1}^{n} \alpha_i \tag{7}$$

Where, δ is the penalty coefficient, whose value is set to 200; $\sum_{i=1}^{n} \alpha_i$ is the sum of penalty coefficients for each path point in a path.

To convert the multi-objective optimization problem into a single-objective problem, the two evaluation indices mentioned above are combined using a weighted sum. The resulting total path evaluation value indicates the quality of the planned path, with smaller values indicating higher quality. Using it as a path fitness value function, considering the actual length and safety of the path comprehensively, it can well ensure that the IPSO algorithm finally gets a safe and short path. The calculation is shown in formula (8).

$$f = \lambda_1 f_1 + \lambda_2 f_2 \tag{8}$$

Where, λ_1、λ_2 is the shortest path weight and collision risk weight. A larger λ_1 indicates that path planning focuses more on shorter path. A larger λ_2 indicates that path planning focuses more on safety. By modifying the weights based on various experimental scenarios, the robot's planned path can be optimized to satisfy its requirements in terms of two evaluation indicators. In this paper, λ_1 and λ_2 are taken as 0.5.

C. Improved Particle Swarm Optimization

Due to the convenience and ease of implementation in adjusting parameters, the PSO has been extensively used in solving optimization problems [3]. Nevertheless, its effectiveness may be limited when dealing with complex search spaces. The primary reason for this is its inclination to converge to local optima and has limited ability to realize balanced evolutionary search [2], [8]. The role of the first part of formula (1) is to endow particles with the capacity to explore new areas, that is, to have the global search ability. If there is no first part, the velocity of the particles is determined solely by their current position and their best historical position. Assuming that particle *i* is initially at the global best position, it will remain stationary until some other particle takes over. Meanwhile, each other particle will move towards the weighted centroid of their own best position and the global best position of the entire population. Therefore, the search process of the PSO without the first part is a process of statistical narrowing of the search area generation by generation. The PSO only has the opportunity to find a solution when the global optimum is within the initial search space. The final solution depends largely on the initial population, and thus, it only has local search capability without the first part. Therefore, ω in formula (1) plays a role in balancing global search and local search. Increasing the value of inertia weights can aid in exploring the entire search space, thereby facilitating global exploration. Conversely, decreasing the value of inertia weight proves helpful in exploring the local search space, facilitating the refinement of the present search region. The appropriate value of the inertial weight ω typically balances the global and local exploration capabilities, thus reducing the quantity of iterations needed to find the ideal solution. Various circumstances necessitate a distinct equilibrium between the ability to explore locally and globally. With this in mind, this paper defines ω as:

$$\omega = 1 + \frac{d}{4 + d} \tag{9}$$

$$d = min\left[||x_i - X_{obs}| - R|, ||y_i - Y_{obs}| - R|\right] \tag{10}$$

Where d represents the distance between particle i and the nearest obstacle; $(X_{obs},\ Y_{obs})$ is the central coordinate of the obstacle in the map; R is the radius of the obstacle.

According to formula (9), ω is larger when particles are in an empty environment. At this point, the search task is more focused on the exploration in the early stage, which is mainly focused on global search, to unearth favorable domains in the realm and steer clear of the algorithm getting trapped in local optima. ω is smaller when particles are in a narrow or complex environment with multiple obstacles nearby. Evolutionary search is gradually shifting towards development, which focuses on local search to avoid particles crossing problem specific constraints and obtain more refined solutions.

This paper aims to optimize the path planning of mobile robots based on an improved particle swarm algorithm, with the shortest accessible path from start to finish as the optimization objective. The main steps are shown in Fig. 2 .

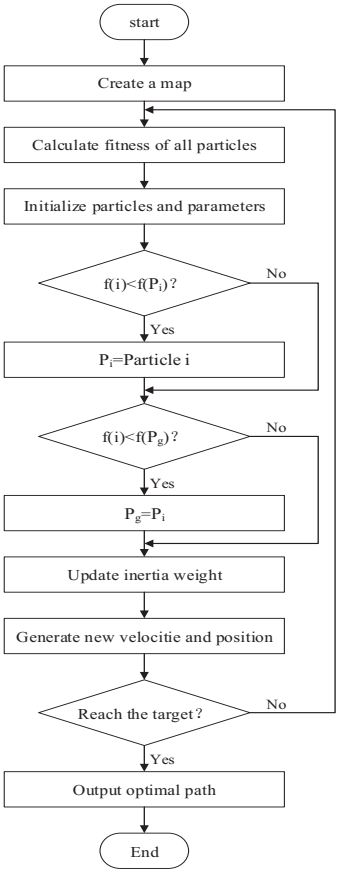


Fig. 2. The IPSO path planning flow chart

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 06:41:09 UTC from IEEE Xplore. Restrictions apply.

It can be summarized as follows:

Step 1: Build the simulation map and determine the starting point S and target point G of the path.

Step 2: The position x and velocity v of each particle in the particle swarm are initialized randomly, while limiting the maximum velocity of particles $V_{max}=0.1(X_{max} - X_{min})$.

Step 3: Calculate the fitness value of each particle according to formula (8).

Step 4: Compare the current fitness value of each particle with the historical best fitness value. If the current value is smaller than the historical best, both the current particle position and fitness value are added to the historical best.

Step 5: Compare the current fitness value of each particle with the global best fitness value. If the current value is smaller than the global best, both the current particle position and fitness value are added to the global best

Step 6: Update the inertia weight ω according to formula (9).

Step 7: Update the velocity v and position x of each particle according to formula (1) and (2).

Step 8: Termination condition evaluation. If the particle reaches the target point G, terminate the iteration and present the most efficient pathway; Alternatively, proceed to step 3.

IV. Experiments

To verify the feasibility and effectiveness of the algorithm, this paper conducted multiple experiments in different simulation environments and compared them with the path planning obtained by traditional PSO. The experimental environment is a 64-bit WIN10 operating system with 8GB of running memory, and the experimental platform is MATLAB R2018b. Table I presents the algorithm parameters utilized in the experiments to satisfy the convergence and stability requirements of the algorithm. The inertia weight ω of the traditional PSO algorithm is set as the average value of the inertia weight of ipso in the experimental process. The starting point and ending point of path planning are set as (0, 0) and (10, 10) respectively.

TABLE I. Algorithm Parameter

N	Max interations	c_1、c_2	V_{max}	ω_{max}
50	200	2	50	1.5

The mathematical portrayal of the robot's operational surroundings for path planning is a crucial element for discussion. It is assumed that: 1) The robot moves in a finite two-dimensional space; 2) The quantity and configuration of obstructions are known, and their height can be disregarded; 3) A robot can be seen as a particle without width.

A. The Sparse Obstacle Environment

First, the study is conducted within a sparsely obstructed environment, and the path planning outputs of the IPSO and the traditional PSO are compared and analyzed. Environment 1 is a stationary map encompassing six circular obstacles, and the domain values of x and y are [0,10]. The objective is to identify a set of path points on the grid-based map that can enable the robot to pass safely and finally reach its final destination, and the path formed by this set is the shortest of all paths that can reach the target location.

Fig. 3 displays the outcomes of the experiment. The black circle in the figure is a static obstacle. The robot's starting point and target point are denoted by S and G respectively. The algorithm charts the planned path as a solid blue line. The left figure is the path planned by the IPSO, while the path on the right follows the path planning approach of the traditional PSO. Fig. 3 illustrates that in environment 1, both algorithms are successful in devising obstacle-free routes from the initial to the final point.

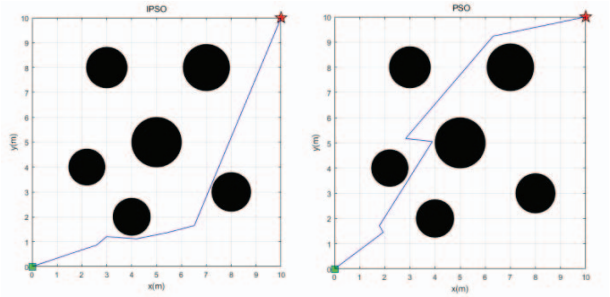


Fig. 3. The optimal path of the IPSO and PSO

To further analyze the specific characteristics of the algorithms, the IPSO and the PSO algorithm conducted 20 independent experiments in environment 1, and obtained the average path length, average number of iterations, and average iteration time, as shown in Table II. It can be seen that when there are fewer obstacles in the map, this algorithm exhibits a slightly superior performance compared to conventional algorithms with respect to both path length and step size, and the convergence velocity of the two algorithms is basically the same.

TABLE II. Comparison of Convergence Results of the IPSO and PSO

Results / Algorithm	Average path length	Average time	Average iterations
IPOS	**14.675**	**5.87**	**5.85**
POS	14.786	5.98	5.90

B. The Dense Obstacle Environment

To further verify the performance of the algorithm, an increase in the number of obstacles present on the map was implemented to examine IPSO's capability in devising a feasible path in a complex environment. The number of obstacles is twenty in environment 2, other parameters remain unchanged. Then simulation test is carried out to obtain the planned path, as shown in Fig. 4.

The experiment was also repeated 20 times in this environment. The average path length, average number of iterations and average iteration time are obtained, as shown in Table III. It can be seen that that when there are many obstacles in the map and the environment is more complex, the safe path searched by the IPSO is shorter in length and has fewer iterations compared to the path planned by the traditional PSO.

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 06:41:09 UTC from IEEE Xplore. Restrictions apply.

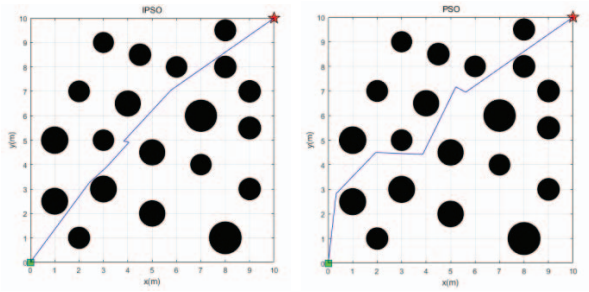


Fig. 4. The optimal path of the IPSO and PSO

TABLE III. COMPARISON OF CONVERGENCE RESULTS OF THE IPSO AND PSO

Results / Algorithm	Average path length	Average time	Average iterations
IPOS	**14.975**	**12.87**	**5.50**
POS	16.786	12.98	6.15

The experimental results obtained from the above simulation in two environments show that the IPSO proposed in this paper can generate a safer and shorter global path at a quicker pace than traditional PSO, and it has more obvious advantages in complex environments. This is because under the same parameter settings, the traditional PSO algorithm has no heuristic information, resulting in suboptimal search performance. In addition, as the number of iterations rises, particle velocity will gradually decrease, making it more susceptible to getting trapped in local optimization, resulting in the inability to obtain a global optimal solution for path planning. This research paper suggests the IPSO algorithm, which dynamically adjusts the inertia weight according to the position of particles in the environment, making the search more purposeful and efficient under the same environment and parameter configuration, avoiding the occurrence of local optima and invalid paths.

V. CONCLUSION

This paper proposes a global path planning method for robots based on the IPSO algorithm. The characteristic of the IPSO is that in the process of continuous iteration of the population, the distance between particles and obstacles is used as the feedback signal of the algorithm to dynamically modify the inertia weight to balance the search demand of the population for the overall or partial regions of the search space. The primary objective of this approach is to address the issues of local optimization and sluggish convergence often encountered by traditional PSO algorithms during path planning. The IPSO algorithm maintains the simple structure and fast convergence velocity of the PSO algorithm, improves the diversity of population, expands the search space, and has low computational complexity. In addition, in order to make the planned path safer and more suitable for robot movement, a fitness function considering path length and collision risk is established. The algorithm proposed in this paper has been evaluated in two distinct environments to assess its performance. The experimental findings reveal that the algorithm has efficiently accomplished the task of collision-free path planning in a static environment featuring multiple obstacles. The study verified the practicability and effectiveness of the approach, highlighting that the IPSO algorithm surpasses the traditional PSO algorithm regarding optimization accuracy and speed.

In real-world environments, the presence of mobile obstacles can pose a significant safety risk for robots. However, the focus of this paper is primarily on the path planning problem within a static, two-dimensional spatial environment with the implementation of the IPSO algorithm, lacking the experiment of the actual physical environment and the treatment of dynamic obstacles. In the future, we will improve the algorithm in this paper, enabling it to effectively designate a path in the dynamic workspace, and test the algorithm on the physical prototype to enhance its application ability in the actual environment. There are many path planning methods for robots, each of which has its advantages and disadvantages. In the future, we will also consider combining genetic algorithms, neural networks, and PSO algorithms to form a novel intelligent path planning approach. This will help enhance the accuracy of robot path planning in obstacle avoidance, speeding up the planning process, and catering to the demands of real-world applications.

ACKNOWLEDGEMENTS

Funding for this study was provided by National Nature Science Foundation of China No.62076014, Beijing Municipal Science & Technology Commission , Administrative Commission of Zhongguancun Science Park No. Z221100000222016.

REFERENCES

[1] J. Escobar, G. Caiza, C. A. Garcial, P. Ayala, and M. V. Garcia, “Applications of Artificial Intelligence Techniques for trajectories optimization in robotics mobile platforms,” Procedia Computer Science, vol. 217, pp. 543-551, 2023.

[2] Y. Liu, “A Laser Intensity Based Autonomous Docking Approach with Application to Mobile Robot Recharging in Unstructured Environments,” In 2022 IEEE 18th International Conference on Automation Science and Engineering, vol. 2, pp. 1297-1302, August, 2022.

[3] S. S. Ge and Y. J. Cui, “Dynamic motion planning for mobile robots using potential field method,” Autonomous Robots, vol. 13, no. 3,pp. 207-222, 2002.

[4] C. DU and B. TANG, “Novel unconventional-active-jamming recon-gnition method for wideband radars based on visibility graphs,” Sensors, vol. 19, no. 10, pp. 234-248, 2019.

[5] Y. F. LIN, Y. J. CHEN, and B. W. HE, “Motion planning method of mobile robot without collision detection RRT*,” Chinese Journal of Scientific Instrument, vol. 41, no. 10, pp. 257-267, 2020.

[6] F.OUYANG “Research on port logistics distribution route planning based on artificial fish swarm algorithm,” Journal of Coastal Research, vol. 115, pp. 78-80, 2020.

[7] H. SHIRI, J. PARK, and M. BENNIS, “Massive autonomous UAV path planning: A neural network based mean-field game theoretic approach,” IEEE Global Communications Conference. IEEE, pp. 1-6, 2019

[8] J. Huiqun, W. Zhonghui, H. Xin, Z. Lei, H. Jiawei, and M. Zhiya, “Path Planning Based on Improved Particle Swarm Optimization Algorithm,” Transactions of the Chinese Society for Agricultural Machinery, vol. 49, no. 12, pp. 371-377, 2018.

[9] S. SHAO, Y. PENG, and C. HE, “Efficient path planning for UAV formation via comprehensively improved particle swarm optimization,” ISA Transactions, vol. 97, pp. 415-430, 2020.

[10] B. K. OLEIWI, H. ROTH, and B. I. KAZEM, “A Hybrid approach based on ACO and Ga for multi objective mobile robot path planning,” Applied Mechanics and Materials, vol. 27, pp. 203-212, 2014.

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 06:41:09 UTC from IEEE Xplore. Restrictions apply.